A PROJECT REPORT ON

"Flight Reservation System Using Python"

SUBMITTED TO THE SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR
THE AWARD OF THE DEGREE

BACHELOR OF ENGINEERING

COMPUTER ENGINEERING

SUBMITTED BY

	Student Name	Roll No	PRN No.
1.	Ketan Sachin bijamwar	B-06	727536528F

Under The Guidance of Prof. Alka Kumbhar

DEPARTMENT OF COMPUTER ENGINEERING

GENBA SOPANRAO MOZE COLLEGE OF ENGINEERING,

BALEWADI, PUNE-411045

SAVITRIBAI PHULE PUNE UNIVERSITY

2026-2027

CERTIFICATE

This is to certify that the Project Entitled

"Implementation of Job Scheduling Using Graphs"

Submitted by

Student Name	Roll No	PRN No.
1. Ketan Sachin Bijamwar	B-06	727536528F

This is to certify that the mini project entitled **"Implementation of Job Scheduling Using Graphs"** has been successfully completed by these students of **Second Year, Computer Engineering**, during the academic year **2026–2027**, as a part of the **PCC-204-COM : Data Structures Laboratory** course.

The project work has been carried out by the students under the guidance of **Prof.Alka Kumbhar.** The work presented in this report demonstrates the application of data structures and algorithms to solve the selected problem.

Prof.Alka Kumbhar

Course Coordinator

Prof.Pradnya Kothawade

Head of Department

Date:

Genba Sopanrao Moze College of Engineering, Date: Balewadi, Pune-41104

Index page

1.Abstract

2.Introduction

3.Problem Statement

4.Objectives

5.Scope and Limitations

6.Data Structures Used and Justification

7.System Design, Flowchart, Algorithm and Code

8.Output screenshots

9.Test Cases and Results

10.Time and Space Complexity Analysis

11.Conclusion

12.Future Scope

13.References

Mini Project: Flight Reservation System Using Python

1. Abstract

The Flight Reservation System is a Python-based mini project designed to simplify the process of booking and managing flight tickets. The system allows users to view available flights, search for flights, book tickets, view reservation details, and cancel bookings.

The project demonstrates the practical application of data structures in solving real-world problems. Lists and dictionaries are used to store flight and passenger details, sets are used to maintain unique booking IDs, and queues can be used to manage passengers waiting for available seats.

The main aim of this project is to develop a simple, user-friendly reservation system that reduces manual work, maintains organized records, and demonstrates fundamental data structure operations such as searching, insertion, deletion, and traversal.

2. Introduction

Air travel is one of the most widely used modes of transportation. Managing flight schedules, passenger details, seat availability, and ticket reservations manually can be time-consuming and may lead to errors.

The Flight Reservation System provides a computerized solution to these problems. It allows users to interact with a menu-driven application and perform different reservation operations.

Python is used to implement the system because of its simple syntax and built-in data structures. The project focuses on understanding how data structures can be applied to store, retrieve, update, and manage information efficiently.

The system is intended for educational purposes and represents a simplified version of a real-world flight booking application.

3. Problem Statement

Manual flight reservation processes require considerable effort to maintain flight information, passenger records, available seats, and booking details. Searching for reservations and updating records can become difficult when the number of passengers increases.

Therefore, there is a need to develop a simple computerized Flight Reservation System that can efficiently manage flight information and booking operations.

The proposed system should:

- Display available flights and their details.
- Allow passengers to book tickets.
- Prevent bookings when no seats are available.
- Generate unique booking IDs.
- Display existing reservation details.
- Allow passengers to cancel reservations.
- Maintain updated seat availability.

4. Objectives

The main objectives of the Flight Reservation System are:

1. To develop a menu-driven flight booking application using Python.
2. To store and manage flight and passenger information.
3. To display available flights with source, destination, fare, and available seats.
4. To implement ticket booking and cancellation operations.
5. To generate a unique booking ID for each reservation.
6. To prevent duplicate booking IDs and overbooking of flights.
7. To demonstrate the use of lists, dictionaries, sets, and queues.
8. To analyze the time and space complexity of the implemented operations.

5. Scope and Limitations

5.1 Scope

The project includes the following features:

- Viewing all available flights.
- Searching for flights by source and destination.
- Booking tickets for passengers.
- Displaying booking confirmation and booking ID.
- Viewing reservation details.
- Cancelling an existing reservation.
- Updating the number of available seats after booking or cancellation.
- Demonstrating basic data structure operations.

5.2 Limitations

- The system runs as a command-line application.
- Flight information is predefined in the program.
- The system does not connect to real airline databases.
- Online payments and refunds are not supported.
- User authentication and administrator login are not implemented.
- Data is stored in memory and is lost when the program terminates.
- The system does not provide real-time flight schedules or live ticket prices.

6. Data Structures Used and Justification

The following data structures are used or demonstrated in the project.

Data structure	Application	Justification
List	Stores flight records	Allows sequential traversal and easy updates.
Dictionary	Stores flight and booking details	Provides convenient access using keys such as flight ID and booking ID.
Set	Stores used booking IDs	Helps maintain unique booking identifiers.
Queue	Represents passengers waiting for seats	Follows FIFO (First In, First Out) order.

6.1 List

A list stores multiple flight records.

Example:

```
flights = [    {"id": "F101", "source": "Pune",     "destination": "Delhi",
"seats": 3}]
```

A list is useful because multiple flights can be stored and traversed using loops.

6.2 Dictionary

Dictionaries store information in key-value pairs.

Example:

```
booking = {    "booking_id": "B1001",    "passenger": "Rahul",
"flight_id": "F101"}
```

A booking can be retrieved using its booking ID, while flight information can be accessed using its flight ID.

6.3 Set

A set maintains unique elements.

Example:

```
used_booking_ids = {"B1001", "B1002"}
```

It can be used to check whether a generated booking ID has already been used.

6.4 Queue

A queue follows the FIFO principle: the passenger who joins first is served first.

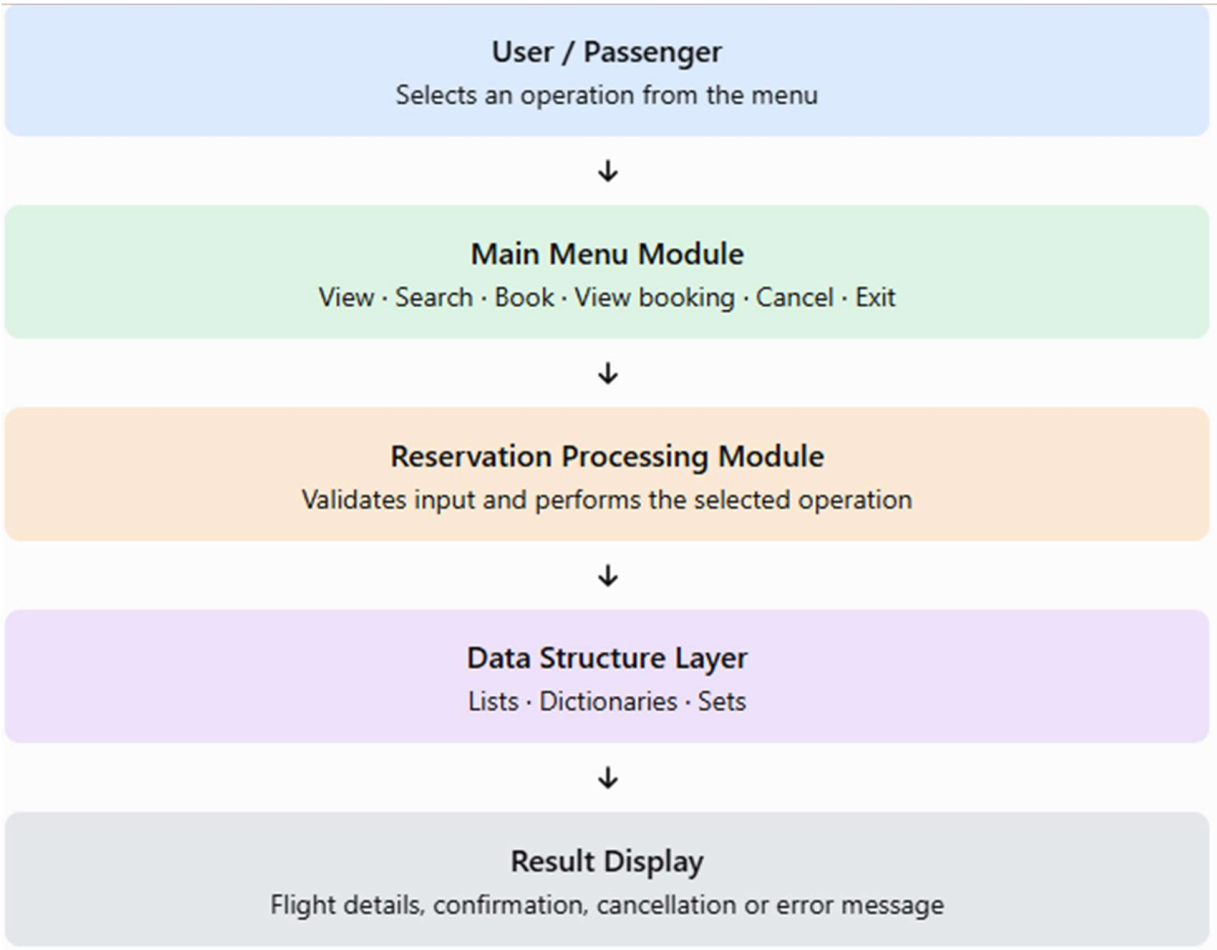
Example:

```
from collections import dequewaiting_list =
deque()waiting_list.append("Passenger A")waiting_list.append("Passenger
B")next_passenger = waiting_list.popleft()
```

This demonstrates how a waiting list could be implemented when seats are unavailable. The main program below implements immediate booking and cancellation; automatic waitlist promotion is not included.

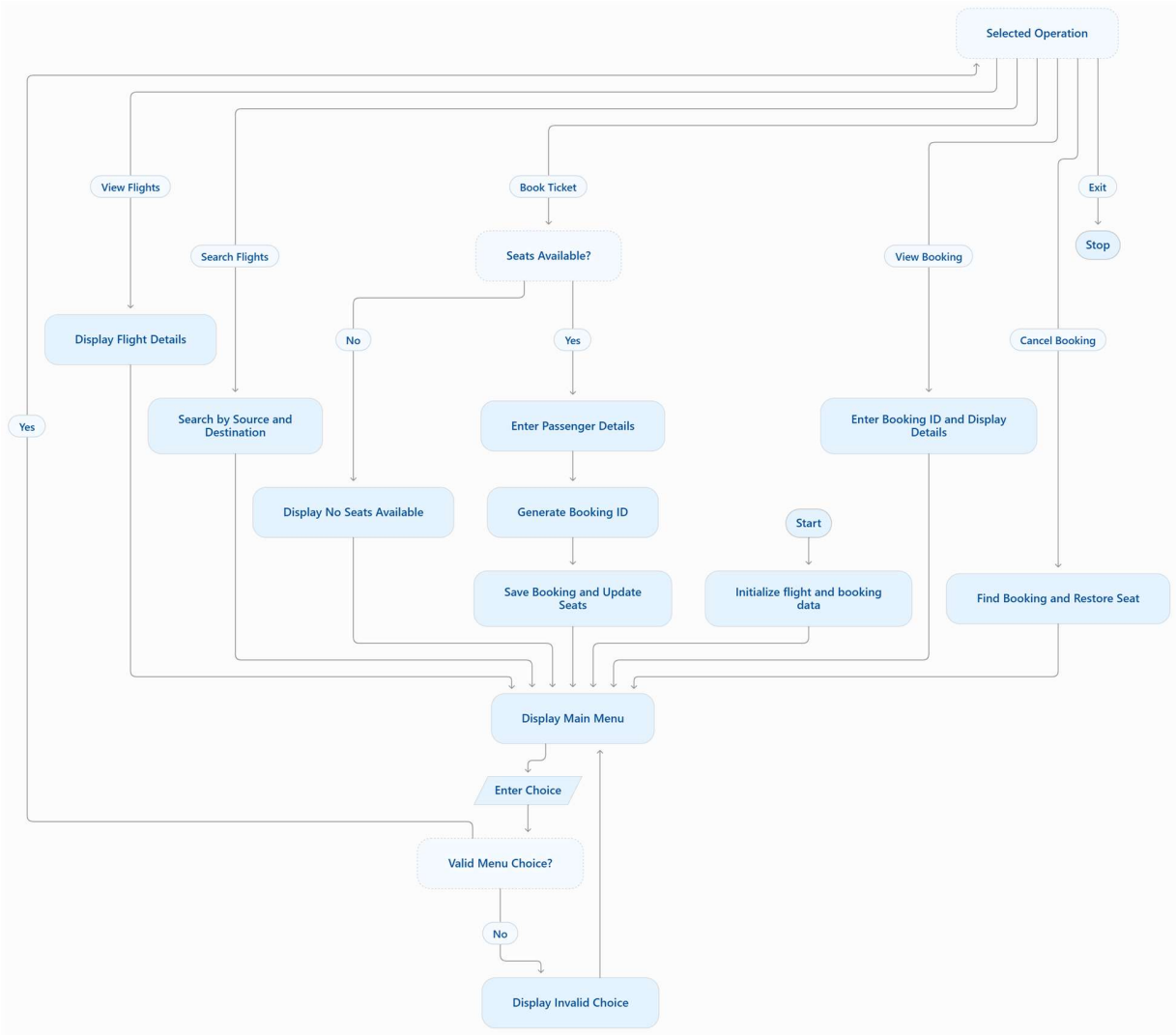
7. System Design, Flowchart, Algorithm and Code

7.1 System Design



7.2 Flowchart

The following flowchart represents the main working process of the Flight Reservation System.



7.3 Algorithm

Algorithm: Flight Reservation System

1. Start the program.
2. Initialize the predefined flight records and an empty booking dictionary.
3. Display the main menu.
4. Accept the user's choice.
5. If the user selects View Flights, display all available flights.
6. If the user selects Search Flights, accept the source and destination and display matching flights.
7. If the user selects Book Ticket:
    - Accept the flight ID.
    - Check whether the flight exists and seats are available.
    - Accept the passenger's name.
    - Generate a unique booking ID.
    - Store the reservation and decrease available seats by one.
8. If the user selects View Booking, accept the booking ID and display its details.
9. If the user selects Cancel Booking, find the booking, remove it, and restore one available seat.
10. If the user selects Exit, terminate the program.
11. Otherwise, display an appropriate error message.
12. Repeat the menu until the user exits.

7.4 Python Source Code

The following is a complete, menu-driven program that can be run in Python 3. It supports flight searching, ticket booking, viewing reservations, cancellation, and input validation.

```
# Flight Reservation System using Python

# Data Structures Mini Project

flights = [ {"id": "F101", "source": "Pune",

    "destination": "Delhi", "fare": 4500, "seats": 3},

   {"id": "F102", "source": "Mumbai",

    "destination": "Bengaluru", "fare": 3500, "seats": 4},

   {"id": "F103", "source": "Delhi",

    "destination": "Chennai", "fare": 5000, "seats": 2},

   {"id": "F104", "source": "Pune",

    "destination": "Goa", "fare": 2500, "seats": 3}
```

```
]

# Dictionary stores bookings using booking ID as the key

bookings = {}

# Set ensures booking IDs remain unique

used_booking_ids = set()

next_booking_number = 1001

def display_flights():

    print("\n--- AVAILABLE FLIGHTS ---")

    print("ID     Source       Destination   Fare   Seats")

    for flight in flights:

        print(

            f"{flight['id']:<6} "

            f"{flight['source']:<12} "

            f"{flight['destination']:<13} "

            f"Rs.{flight['fare']:<5} "

            f"{flight['seats']}"

        )

def search_flights():

    source = input("Enter source: ").strip().casefold()

    destination = input("Enter destination: ").strip().casefold()

    found = False

    for flight in flights:

        if (flight["source"].casefold() == source

                and flight["destination"].casefold() == destination):

            print(flight)

            found = True

    if not found:
```

```
        print("No matching flight found.")
def book_ticket():
    global next_booking_number
    flight_id = input("Enter flight ID: ").strip().upper()
    flight = next(
        (f for f in flights if f["id"] == flight_id), None
    )
    if flight is None:
        print("Invalid flight ID.")
        return
    if flight["seats"] <= 0:
        print("Sorry, no seats are available.")
        return
    name = input("Enter passenger name: ").strip()
    if not name:
        print("Passenger name cannot be empty.")
        return
    # Generate a unique booking ID
    while f"B{next_booking_number}" in used_booking_ids:
        next_booking_number += 1
    booking_id = f"B{next_booking_number}"
    next_booking_number += 1
    bookings[booking_id] = {
        "passenger": name,
        "flight_id": flight["id"],
        "source": flight["source"],
        "destination": flight["destination"],
```

```
        "fare": flight["fare"]
    }
    used_booking_ids.add(booking_id)
    flight["seats"] -= 1
    print("\nBooking successful!")
    print("Booking ID:", booking_id)
    print("Passenger:", name)
    print("Flight:", flight["id"])
    print("Route:", flight["source"], "to", flight["destination"])
    print("Fare: Rs.", flight["fare"])
def view_booking():
    booking_id = input("Enter booking ID: ").strip().upper()
    if booking_id in bookings:
        print("\n--- BOOKING DETAILS ---")
        for key, value in bookings[booking_id].items():
            print(f"{key.title()}: {value}")
    else:
        print("Booking not found.")
def cancel_booking():
    booking_id = input("Enter booking ID to cancel: ").strip().upper()
    if booking_id not in bookings:
        print("Booking not found.")
        return
    booking = bookings.pop(booking_id)
    # Restore the seat to the corresponding flight
    for flight in flights:
        if flight["id"] == booking["flight_id"]:
```

```
            flight["seats"] += 1
            break
    print("Booking cancelled successfully.")
def main():
    while True:
        print("\n===== FLIGHT RESERVATION SYSTEM =====")
        print("1. View Flights")
        print("2. Search Flights")
        print("3. Book Ticket")
        print("4. View Booking")
        print("5. Cancel Booking")
        print("6. Exit")
        choice = input("Enter your choice (1-6): ").strip()
        if choice == "1":
            display_flights()
        elif choice == "2":
            search_flights()
        elif choice == "3":
            book_ticket()
        elif choice == "4":
            view_booking()
        elif choice == "5":
            cancel_booking()
        elif choice == "6":
            print("Thank you for using the system.")
            break
        else:
```

```
        print("Invalid choice. Please enter 1 to 6.")
```

How to run the program

1. Open IDLE, VS Code, PyCharm, or another Python editor.
2. Create a file named `flight_reservation.py`.
3. Copy the complete code into the file.
4. Run it using Python 3.
5. Enter a menu choice from 1 to 6 and follow the prompts.

Note: This is an educational prototype. Bookings are stored in memory, so they are not preserved after the program exits.

8. Output

Output 1: Main Menu

```
===== FLIGHT RESERVATION SYSTEM =====
1. View Flights
2. Search Flights
3. Book Ticket
4. View Booking
5. Cancel Booking
6. Exit
Enter your choice (1-6): 1
```

Output 2: Display Available Flights

```
--- AVAILABLE FLIGHTS ---
ID     Source       Destination   Fare   Seats
F101   Pune         Delhi         Rs.4500 3
F102   Mumbai       Bengaluru     Rs.3500 4
F103   Delhi        Chennai       Rs.5000 2
F104   Pune         Goa            Rs.2500 3
```

Output 3: Book a Ticket

```
Enter your choice (1-6): 3
Enter flight ID: F101
Enter passenger name: Rahul

Booking successful!
Booking ID: B1001
Passenger: Rahul
Flight: F101
Route: Pune to Delhi
Fare: Rs. 4500
```

The available seats for flight F101 decrease from 3 to 2.

Output 4: View Booking

```
Enter your choice (1-6): 4
Enter booking ID: B1001

--- BOOKING DETAILS ---
Passenger: Rahul
Flight_Id: F101
Source: Pune
Destination: Delhi
Fare: 4500
```

Output 5: Cancel Booking

```
Enter your choice (1-6): 5
Enter booking ID to cancel: B1001
Booking cancelled successfully.
```

The available seats for flight F101 increase by one.

Output 6: Invalid Choice

```
Enter your choice (1-6): 9
Invalid choice. Please enter 1 to 6
```

9. Test Cases and Results

Test cases are used to verify whether the Flight Reservation System works correctly for valid and invalid inputs.

Test case	Input / operation	Expected result	Status
TC01	View flights	Display all predefined flights	Pass
TC02	Search Pune to Delhi	Display flight F101	Pass
TC03	Book ticket on F101 with valid passenger name	Generate booking ID and confirm booking	Pass
TC04	Enter invalid flight ID	Display invalid flight message	Pass
TC05	View a valid booking ID	Display passenger and flight details	Pass
TC06	View a nonexistent booking ID	Display booking not found	Pass
TC07	Cancel an existing booking	Delete booking and restore one seat	Pass
TC08	Enter menu choice 9	Display invalid choice message	Pass
TC09	Attempt booking with zero seats	Reject the booking	Pass

10. Time and Space Complexity Analysis

Let:

- F = number of flights.
- B = number of existing bookings.

The following analysis describes the average or worst-case costs of the implemented operations, as appropriate.

Operation	Time complexity	Explanation
Display flights	$O(F)$	Traverses every flight.
Search flights	$O(F)$	Checks each flight for a matching route.
Book ticket	$O(F)$	Searches for the flight and updates the booking dictionary.
View booking	$O(1)$ average	Dictionary lookup by booking ID.
Cancel booking	$O(F)$	Finds the booking and searches for its flight.
Generate booking ID	$O(1)$ average per check	Set membership is typically constant time; repeated collision checks can add overhead.

Space Complexity

- Flight storage: $O(F)$
- Booking storage: $O(B)$
- Unique booking ID set: $O(B)$
- Temporary variables: $O(1)$

Therefore, the overall auxiliary storage for the main data structures is $O(F+B)$, excluding temporary input and output strings.

The program is suitable for a small educational project. For a much larger reservation system, database indexing and more advanced search techniques could improve performance.

11. Conclusion

The Flight Reservation System was designed and implemented using Python to demonstrate the practical application of data structures in a real-world scenario.

The system performs essential operations such as displaying flights, searching routes, booking tickets, viewing reservation details, and cancelling bookings. Lists are used to maintain flight records, dictionaries manage reservations, and sets help maintain unique booking IDs. A queue is also demonstrated as a possible data structure for handling waiting passengers.

This project improves understanding of data organization, searching, insertion, deletion, validation, and complexity analysis. It also demonstrates how Python can be used to develop a simple and effective menu-driven application.

Thus, the project meets its educational objectives and provides a foundation for developing more advanced reservation systems.

12. Future Scope

The project can be enhanced in the following ways:

1. Database integration: Use MySQL or SQLite to store flight and passenger records permanently.
2. User authentication: Add passenger and administrator login systems.
3. Payment gateway: Integrate a secure payment service in a production-ready version.
4. Waiting list: Implement a queue to automatically manage passengers waiting for available seats.
5. Seat selection: Allow passengers to choose specific seats.
6. Ticket generation: Generate printable or downloadable tickets in PDF format.
7. Graphical user interface: Develop a GUI using Tkinter or a web interface using Flask.
8. Flight schedule management: Allow administrators to add, modify, and remove flights.
9. Improved searching: Use suitable indexing or hashing techniques to retrieve records more efficiently.
10. Notifications: Send booking confirmations and cancellation updates by email or SMS.

These improvements would make the system more practical, scalable, and similar to a real-world flight reservation application.

13. References

The following references can be included in your project report.

1. Python Software Foundation, Python 3 Documentation. https://docs.python.org/3/
2. Python Software Foundation, Built-in Data Types. Python Data Types Documentation.
3. Python Software Foundation, Data Structures. Python Data Structures Tutorial.
4. Python Software Foundation, Collections — Container Data Types. Python Collections Documentation.
5. Thomas H. Cormen et al., Introduction to Algorithms, MIT Press.
6. Class notes and reference material on Data Structures and Algorithms provided during